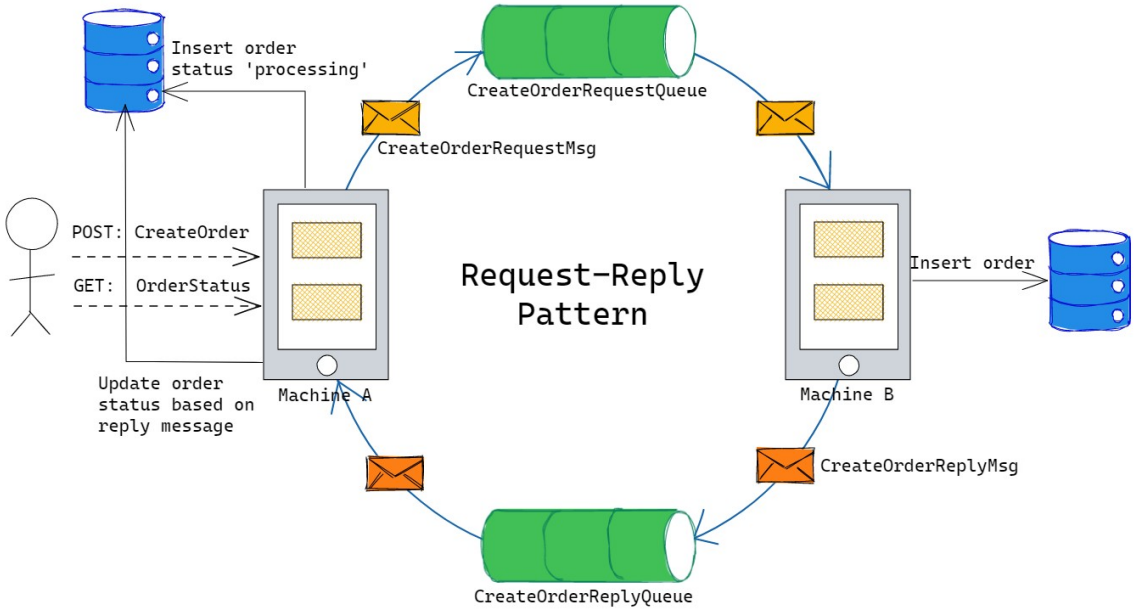
Licenciatura en Ciencias de la Computación – Sistemas Distribuidos

19 de Abril de 2026 – Trabajo Práctico N° 4

Paula Martínez y Adriano Fabris

Patrón Request-Reply

Es un patrón síncrono de comunicación 1:1. Un cliente envía una petición y queda bloqueado hasta recibir una respuesta del servidor. Existe una correspondencia estricta entre request y reply.



0) Ejecutar ambos servidor y clientes.

1) Detener el servidor y ejecutar el cliente, ¿qué pasó?

El cliente se conecta, envía el mensaje, pero se queda indefinidamente esperando un mensaje por parte del servidor, específicamente se queda en:

const [result] = await sock.receive()

2) Ahora ejecutar el servidor, ¿qué pasó con el cliente?

El cliente recibe el mensaje por parte del servidor y continúa su ejecución normalmente.

3) ¿Esto es lo mismo en otros protocolos conocidos?

No, en HTTP si el servidor no contesta la conexión falla.

En ZMQ la conexión queda a la espera de que el servidor se conecte, por mientras el mensaje se pone en una **cola local** del lado del cliente y se intenta enviar (en loop) hasta que el server se conecte, el buffer se llene o el proceso se cierre.

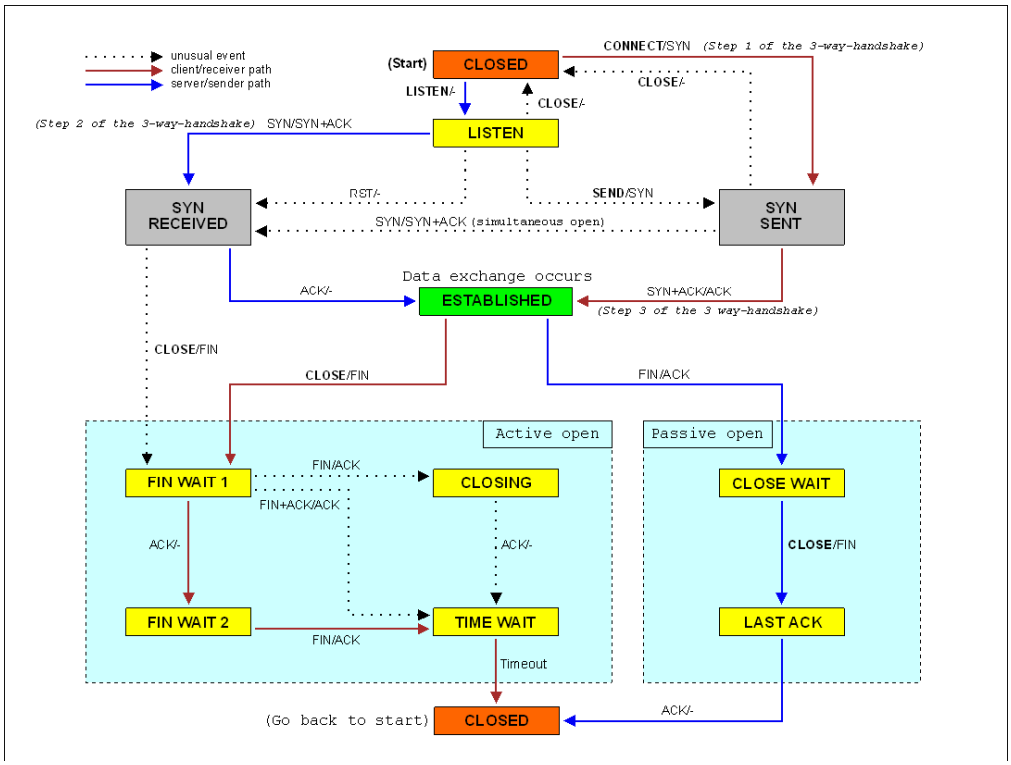
4) ¿Por qué cree que pasa esto?

Creencia: porque hemos definido una sentencia *await sock.receive()*, que espera hasta tanto exista una conexión.

Realidad: tal cual la creencia, pero con su debida razón. La justificación es que ZMQ aplica un nivel más de abstracción para desacoplar envío de recepción y viceversa, el diseño *request-reply* **prioriza tolerancia a fallos**, es decir, no levanta una excepción aún cuando el servidor está desconectado, esto con el fin de que este eventualmente puede fallar.

En código vemos que usamos un objeto *const sock = zmq.Request*, luego, toda esta lógica descripta está encapsulada en tal variable.

5) Investigue cómo listar los puertos abiertos tanto en la máquina que hace de cliente como en la que hace de servidor con solo un programa andando. ¿Cuál es la diferencia entre ambos?



Creo que las transiciones indican I/O respecto del color.

Por ejemplo desde SYNSENT a ESTABLISHED tenemos SYN+ACK/ACK por parte del cliente, o sea, el cliente recibe SYN+ACK y envía ACK al servidor.

In TCP, a connection is a state machine. Key states:

- LISTEN: **server** waiting for incoming connections.
- SYN-SENT: **client** sent SYN, waiting for SYN-ACK.
- SYN-RECEIVED: **server** got SYN, sent SYN-ACK, waiting for ACK.
- ESTABLISHED: connection open, data flows both directions.
- FIN-WAIT-1: one side sent FIN (wants to close), waiting ACK.
- FIN-WAIT-2: FIN acknowledged, waiting for peer's FIN.
- CLOSE-WAIT: received FIN, waiting for local app to close.
- LAST-ACK: sent FIN after receiving FIN, waiting final ACK.
- TIME-WAIT: waits (~2×MSL) after close to ensure delayed packets are discarded.
- CLOSED: no connection state.

Un **puerto abierto** es aquel que está en **LISTEN**, ya que en ESTABLISHED la conexión está activa, pero esto no implica que el proceso esté escuchando.

<u>Server's side</u>

Podemos usar el comando ss (socket statistics), con el flag -a para listar tanto LISTEN como ESTAB(lished) ports.

Para 3 clientes conectados (usando loop para no cerrar la conexión):

```
debian@caso-3-instancia-1:~/practica-zmq/01 req-rep$ ss -a | grep -e :3000 -e Port
Netid State      Recv-Q Send-Q                                   Local Address:Port                                   Peer Address:Port
tcp   LISTEN     0      100                                            0.0.0.0:3000                                        0.0.0.0:*
tcp   ESTAB      0      0                                           10.66.0.118:3000                                   10.64.17.240:48940
tcp   ESTAB      0      0                                           10.66.0.118:3000                                   10.64.17.240:53680
tcp   ESTAB      0      0                                           10.66.0.118:3000                                   10.64.17.240:50178
```

El server LISTENs en 3000 y a la vez tiene 3 ESTAB, uno por cada cliente.

<u>Clients's side</u>

Como la máquina del cliente es la misma, vemos que tenemos 3 procesos.

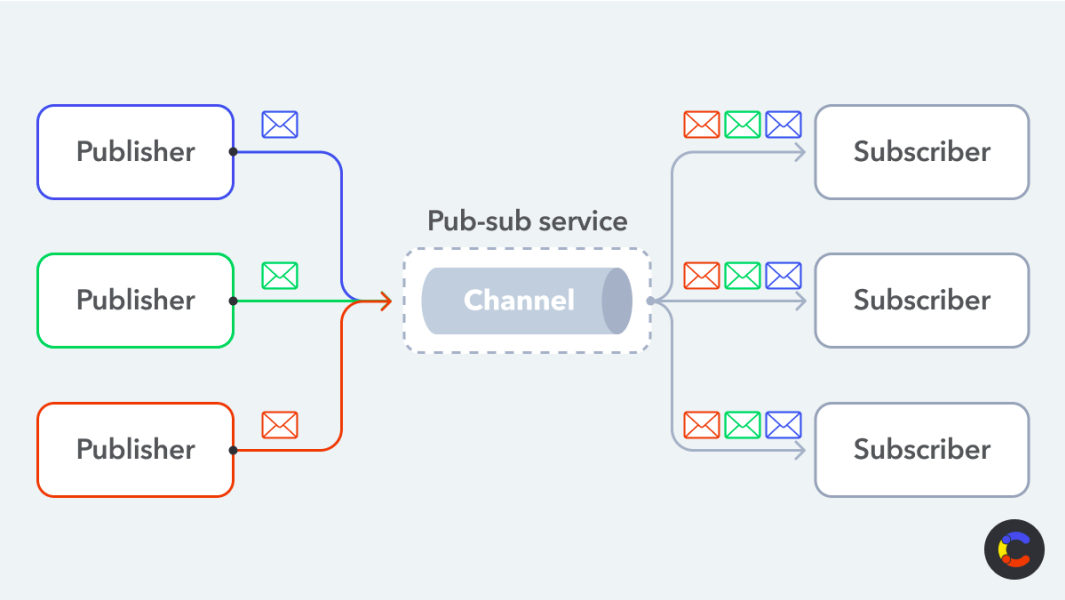
```
adriano@notebook:~$ ss -a | grep -e :3000 -e Port
RTNETLINK answers: Invalid argument
Netid State  Recv-Q Send-Q                    Local Address:Port                 Peer Address:Port  Process
tcp   ESTAB  0      0                          172.29.0.166:48940                 10.66.0.118:3000
tcp   ESTAB  0      0                          172.29.0.166:50178                 10.66.0.118:3000
tcp   ESTAB  0      0                          172.29.0.166:53680                 10.66.0.118:3000
```

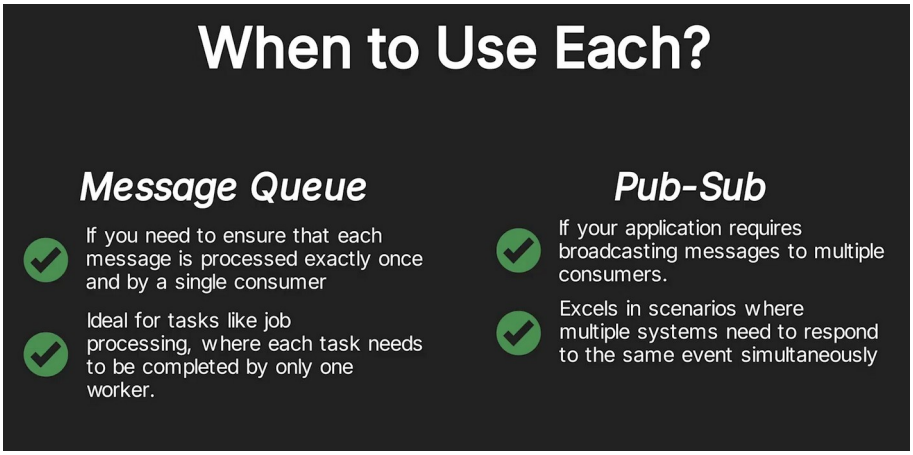
Diferencia: la diferencia radica en que el servidor siempre tiene un proceso en LISTEN (además de los ESTAB), mientras que los clientes sólo tienen el ESTAB respectivo con el server (sockets).

Ejemplos: APIs REST; Microservices RPCs (Remote Procedure Call); DB queries

---

Patrón Publisher-Subscriber





1. Iniciando y deteniendo publisher y subscriber

a. ¿Qué pasa si detengo solo el subscriber?

El publisher sigue publicando mensajes, independientemente de que si lo están escuchando o no.

b. ¿Qué pasa con los mensajes publicados?

Si hay 0 subs, se pierde, ya que si existe al menos 1 sub el mensaje podría de alguna manera retenerse y reenviarse.

c. ¿Y si lo vuelvo a ejecutar el subscriber, que pasó con los mensajes enviados?

Los mensajes enviados en el período en el que el subscriber no estaba suscrito se pierden, por lo tanto el suscriptor comienza a recibir a partir del momento en el que se conecta.

d. ¿Qué pasa si detengo solo el publisher?

El suscriptor se queda esperando una publicación.

e. Y si vuelvo a ejecutar el publisher después de detenerlo, ¿que pasa en el subscriber?

El subscriber continúa recibiendo como si nada hubiese pasado en el medio.

f. ¿Hay algo que te llame la atención?

No. Simplemente es una lógica diferente a la del caso anterior, en el que el mensaje no se perdía si alguna de las partes se desconectaba.

Aquí la lógica es como la del transporte público, cuando el autobús pasa (publisher publica), si uno (suscriber) no se sube (no se suscribe), perdemos el autobús, tendremos que esperar al próximo. Y si el autobús no pasa (publisher inactivo/desconectado) el pasajero no tiene cómo transportarse.

Otra analogía, con la TV en vivo: si no estás conectado, perdés los mensajes.

<u>2. Probando múltiples procesos</u>

a. Probá ejecutando dos o más subscribers en dos consolas distintas. ¿Funciona?

Funciona correctamente. Todos reciben los mismos mensajes.

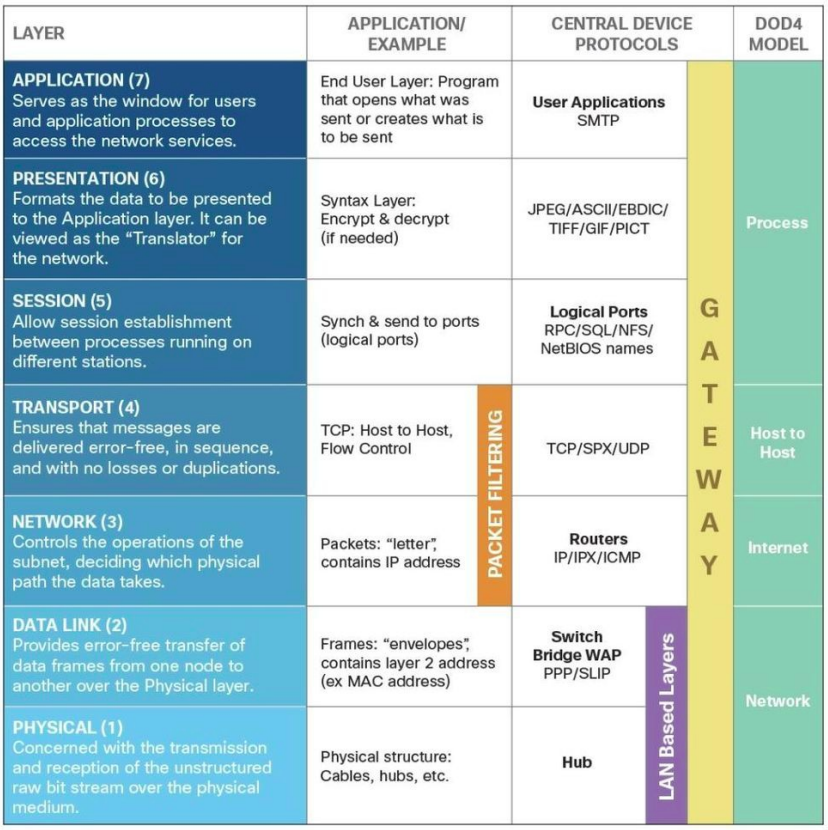
b. Ahora intenta probar ejecutar dos publisher en el servidor. ¿Funciona? ¿Por qué no?

No funciona. El puerto ya está ocupado.

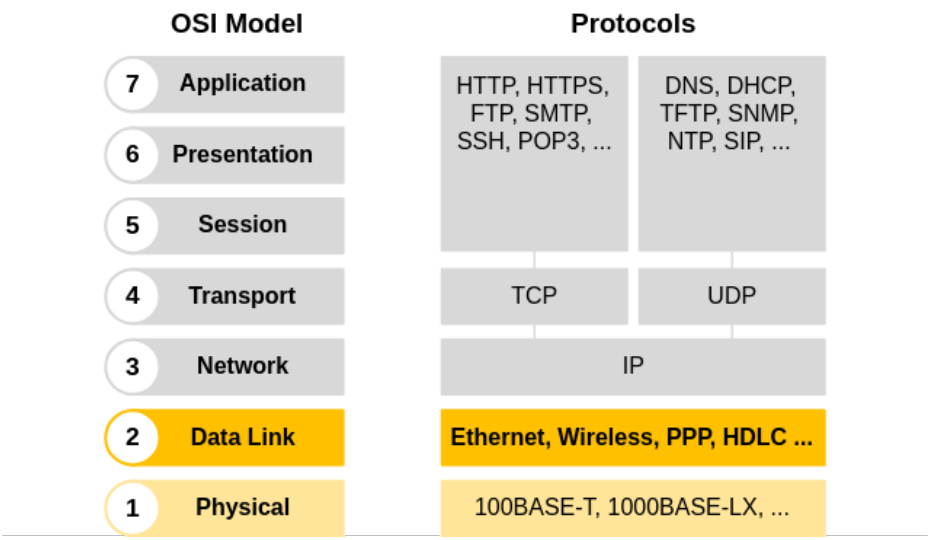
```
debian@caso-3-instancia-1:~/practica-zmq/02 pub-sub$ node publisher.js
node:internal/process/promises:332
    triggerUncaughtException(err, true /* fromPromise */);
    ^

[Error: Address already in use] {
  errno: 98,
  code: 'EADDRINUSE',
  address: 'tcp://*:3000'
}
```

Lanza un error que indica que la dirección ya existe, es decir, el puerto ya está ocupado.
Nota: Lo mismo pasa en el patrón request-reply.

De alguna manera el servidor (ya sea replier o publisher) es un proceso, por lo tanto corre en un puerto, y un puerto identifica a un único proceso.



LAYER	APPLICATION/ EXAMPLE		CENTRAL DEVICE PROTOCOLS			DOD4 MODEL
APPLICATION (7) Serves as the window for users and application processes to access the network services.	End User Layer: Program that opens what was sent or creates what is to be sent		**User Applications** SMTP		GATEWAY	Process
PRESENTATION (6) Formats the data to be presented to the Application layer. It can be viewed as the "Translator" for the network.	Syntax Layer: Encrypt & decrypt (if needed)		JPEG/ASCII/EBDIC/ TIFF/GIF/PICT			
SESSION (5) Allow session establishment between processes running on different stations.	Synch & send to ports (logical ports)		**Logical Ports** RPC/SQL/NFS/ NetBIOS names			
TRANSPORT (4) Ensures that messages are delivered error-free, in sequence, and with no losses or duplications.	TCP: Host to Host, Flow Control	PACKET FILTERING	TCP/SPX/UDP			Host to Host
NETWORK (3) Controls the operations of the subnet, deciding which physical path the data takes.	Packets: "letter", contains IP address		**Routers** IP/IPX/ICMP			Internet
DATA LINK (2) Provides error-free transfer of data frames from one node to another over the Physical layer.	Frames: "envelopes", contains layer 2 address (ex MAC address)		**Switch Bridge WAP** PPP/SLIP	LAN Based Layers		Network
PHYSICAL (1) Concerned with the transmission and reception of the unstructured raw bit stream over the physical medium.	Physical structure: Cables, hubs, etc.		**Hub**			



c. ¿Qué pasa si cambio el tópico en el cliente o en servidor?

En el cliente: si el nuevo tópico no existe en el servidor, el cliente no recibe publicaciones.
En el servidor: si no existe un cliente suscrito a ese nuevo tópico entonces ningún cliente recibe publicaciones.

En conclusión, si no coincide entre publisher y subscriber, no se reciben mensajes.

d. ¿Puede un pub tener más de un tópico en ejecución?

Creencia: sí. ¿Por qué debería estar limitado a sólo uno?
Realidad: creencia correcta.

e. ¿Y un subscriber atender múltiples tópicos?

Creencia: sí. ¿Por qué debería estar limitado a sólo uno?
Realidad: creencia correcta.

Ejemplo de uso real

- sistemas de logs
- notificaciones en tiempo real
- streaming de eventos
- monitoreo

Patrón Push(er)/Pull(ers) o Producer/Workers respectivamente

Un productor envía tareas y múltiples workers las consumen. Cada tarea es procesada por un único worker. Es un patrón de distribución de carga.

Diferencias con PubSub

1. Iniciando y deteniendo publisher y worker

a. ¿Qué pasa si detengo los workers?

Diferente del PubSub: si no hay workers disponibles el publisher queda a la espera de que alguno se conecte.

b. ¿Qué pasa con los mensajes publicados?

Diferente del PubSub: aquí no hay pérdida de mensajes, si una tarea no es procesada debido a que no existen workers, no se acumulan tareas, se espera hasta tanto se conecte un worker y se delege la tarea.

c. ¿Y si lo vuelvo a ejecutar el worker, que pasó con los mensajes enviados?

Diferente del PubSub: Los mensajes enviados en el período en el que el worker no estaba suscrito **NO** se pierden.
Se procesan las tareas penientes.

d. ¿Qué pasa si detengo solo el publisher?

Igual al PubSub: Los workers se quedan esperando tareas.

e. Y si vuelvo a ejecutar el publisher después de detenerlo, ¿que pasa en el worker?

Igual al PubSub: el worker continúa recibiendo como si nada hubiese pasado en el medio.

f. ¿Hay algo que te llame la atención?

No. Simplemente es una lógica diferente.
Aquí los mensajes siempre son despachados a un worker respetando el orden impuesto por el puller.

2. Probando múltiples procesos

a. Probá ejecutando dos o más subscribers en dos consolas distintas. ¿Funciona?

Sí. El reparto de tareas es por Round-Robin.
Hay balanceo de carga: si un worker está ocupado o su cola llena, se omite y se asigna a otro. Los workers más rápidos terminan procesando más tareas.

b. Ahora intenta probar ejecutar dos publisher en el servidor. ¿Funciona? ¿Por qué no?

No, no funciona por el mismo motivo que en los casos anteriores: un puerto identifica unívocamente a un proceso.

Ejemplo de uso real

- procesamiento de imágenes
- colas de trabajo
- master/worker
- sistemas batch
- procesamiento distribuido

Patrón	Tipo	Comunicación	Ventajas	Desventajas	Ejemplos de uso real	Analogía
ReqRep	Síncrono (a nivel protocolo)	1 a 1	Simple, control directo, trazabilidad clara, respuesta garantizada	Bloqueo, acoplamiento fuerte, menor tolerancia a fallos, menor escalabilidad	APIs HTTP, RPC, consultas a bases de datos	Llamada telefónica
PubSub	Asíncrono	1 a N (broadcast)	Desacople, escalable, múltiples consumidores, eficiente para eventos	Pérdida de mensajes, sin confirmación, difícil debugging/tracing	Logs, notificaciones, eventos en tiempo real	Canal de TV / radio
PushPull	Asíncrono	1 a N (distribución)	Balanceo de carga, paralelismo, alto throughput	Sin respuesta directa, requiere canal extra para resultados	Procesamiento batch, colas de trabajo, workers distribuidos	Fila de cajas en supermercado